University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 2	**CLOs to be covered:** CLO1
Lab Title: For Loops	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2

Lab Goals / Objectives:

• Understand the concept and syntax of for loops in Python

• Learn to use the range() function with for loops

• Apply for loops to solve practical problems

• Learn to iterate over strings using for loops

• Understand nested for loops and their applications

• Use for loops with conditional statements (if/else)

• Import and use the random and string modules

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. The for Loop in Python

A for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) and execute a block of code for each item. Unlike other programming languages, Python's for loop is primarily designed for iteration over sequences.

```
for variable in sequence:
    # Code block to execute
```

2. The range() Function

The range() function generates a sequence of numbers. It is commonly used with for loops:

• range(stop) - Generates numbers from 0 to stop-1

• range(start, stop) - Generates numbers from start to stop-1

• range(start, stop, step) - Generates numbers from start to stop-1 with given step

```
for i in range(5):      # 0, 1, 2, 3, 4
for i in range(1, 6):   # 1, 2, 3, 4, 5
for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
```

3. Iterating Over Strings

A string is a sequence of characters, so you can iterate over each character using a for loop:

```
for char in 'Hello':
    print(char)
```

4. The reversed() Function

The reversed() function returns a reversed iterator of a sequence:

```
for digit in reversed('1010'):
    print(digit)  # Prints: 0, 1, 0, 1
```

5. Nested For Loops

A nested for loop is a loop inside another loop. The inner loop executes completely for each iteration of the outer loop:

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

6. The break Statement

The break statement is used to exit a loop prematurely when a certain condition is met:

```
for i in range(10):
    if i == 5:
        break
```

7. String Methods: isalpha()

The isalpha() method returns True if all characters in the string are alphabetic and there is at least one character:

```
'A'.isalpha()   # True
'1'.isalpha()   # False
```

8. The random and string Modules

The random module provides functions for generating random numbers and making random selections. The string module contains useful string constants:

• string.ascii_uppercase - All uppercase letters (A-Z)

• string.ascii_lowercase - All lowercase letters (a-z)

• string.digits - All digit characters (0-9)

• string.punctuation - All special characters

• random.choice(seq) - Returns a random element from a sequence

9. String Concatenation

Strings can be concatenated using the + operator:

```
result = ''
for i in range(3):
    result = result + str(i)
```

10. Prime Numbers

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. To check if a number is prime, we test divisibility from 2 up to number-1.

Lab Work:

Task 1: Binary to Decimal Converter

Write a program that converts a binary number into its decimal equivalent using a for loop.

Code Summary:

This program takes a binary string as input, iterates over each digit in reverse order using reversed(), multiplies each digit by 2 raised to the appropriate power, and accumulates the result to get the decimal equivalent.

Key Code Lines:

```
# LAB TASK 01: Binary to Decimal Converter
binary = input("Enter a binary number: ")
decimal = 0
power = 0
for digit in reversed(binary):
    decimal = decimal + int(digit) * (2 ** power)
    power = power + 1
print("Answer is:", decimal)
```

Output:

Task 2: Count Vowels and Consonants

Write a program that takes a sentence from the user and counts the number of vowels and consonants using a for loop.

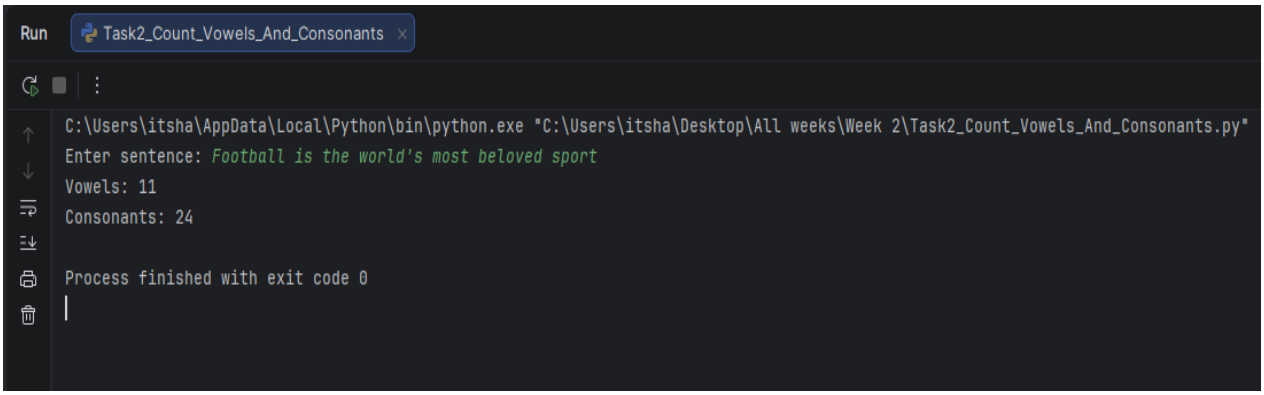
Code Summary:

This program iterates over each character in the input string, checks if it is an alphabetic character using isalpha(), then checks if it exists in the vowels string to classify it as a vowel or consonant.

Key Code Lines:

```
# LAB TASK 02: Count Vowels and Consonants
sentence = input("Enter sentence: ")
vowels = "aeiouAEIOU"
vowel_count = 0
consonant_count = 0
for letter in sentence:
    if letter.isalpha():
        if letter in vowels:
            vowel_count = vowel_count + 1
        else:
            consonant_count = consonant_count + 1
print("Vowels:", vowel_count)
print("Consonants:", consonant_count)
```

Output:



Task 3: Prime Numbers in Range

Write a program that takes a range from the user and prints all prime numbers within that range along with their sum.

Code Summary:

This program uses nested for loops. The outer loop iterates through numbers in the given range, and the inner loop checks divisibility from 2 to number-1. If no divisor is found, the number is prime.

Key Code Lines:

```
# LAB TASK 03: Prime Numbers in Range
start = int(input("Enter start: "))
end = int(input("Enter end: "))
total = 0
for number in range(start, end):
    if number > 1:
        prime = True
        for x in range(2, number):
            if number % x == 0:
                prime = False
                break
        if prime:
            print(number)
            total = total + number
print("Sum is:", total)
```

Output:

Task 4: Random Password Generator

Write a program that asks the user what kind of password they want and generates a random password of specified length.

Code Summary:

This program uses the random and string modules. It collects user preferences for character types (uppercase, lowercase, digits, special), builds a character pool, and randomly selects characters to generate a secure password.

Key Code Lines:

```
# LAB TASK 04: Random Password Generator
import random
import string
length = int(input("Enter length: "))
characters = ""
upper = input("Uppercase? y/n: ")
lower = input("Lowercase? y/n: ")
digit = input("Digits? y/n: ")
special = input("Special? y/n: ")
if upper == "y":
    characters = characters + string.ascii_uppercase
if lower == "y":
    characters = characters + string.ascii_lowercase
if digit == "y":
    characters = characters + string.digits
if special == "y":
    characters = characters + string.punctuation
password = ""
for p in range(length):
    password = password + random.choice(characters)
print("Password is:", password)
```

Output:

Task 5: Diamond Star Pattern

Write a program that prints a diamond-shaped star pattern using nested for loops.

Code Summary:

This program uses nested for loops to create a diamond pattern. The first outer loop prints an increasing triangle, and the second outer loop prints a decreasing triangle, creating a diamond shape.

Key Code Lines:

```
# LAB TASK 05: Diamond Star Pattern
rows = 4
for i in range(1, rows + 1):
    for j in range(i):
        print("*", end=" ")
    print()
for i in range(rows - 1, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output:

Conclusions:

• Successfully understood and applied for loops for iterative problem solving

• Learned to use the range() function with various parameter combinations

• Mastered iterating over strings and using string methods like isalpha()

- Applied nested for loops to generate complex patterns
- Used the break statement to optimize loop execution
- Successfully imported and utilized the random and string modules
- Developed practical applications including number conversion and password generation